

02

# Building Agents

Day 2 | Scalable, low-touch processes

---

Abhimanyu S.

2 hours · Day 2 of 4 · one pipeline, built live on real systems

[abhimanyu@novnex.com](mailto:abhimanyu@novnex.com)

BUILDING AGENTS · A FOUR-SESSION BUILD PROGRAMME

# The programme

*Four sessions, one thread: from one working agent to a governed fleet.*

## DAY 1

### Fundamentals

What an agent is, and where it belongs in a workflow

#### YOU LEAVE ABLE TO

Build one from nothing, and read any other.

## DAY 2

### Processes

Steps, triggers, and a low-touch process that runs without you

#### YOU LEAVE ABLE TO

Design a process an agent can safely run.

## DAY 3

### Testing

Agents that write the tests, catch a contract break, and triage failures

#### YOU LEAVE ABLE TO

Wire three agents into one pipeline.

## DAY 4

### The Ecosystem

Inventory, identity, run records and cost

#### YOU LEAVE ABLE TO

Run agents beyond the team that built them.

YOU ARE HERE

The judge built last session still works — and a person still triggers it, feeds it, and reads it. Today the person leaves the loop.

# What today is

*Mostly a running system. The slides frame it; the terminal, the tracker and the inbox carry it.*

## STANDING FROM DAY 1

- Story Check: judges a story against our Definition of Ready
- Judgement is agent work; a rule is machinery — steps marked
- The funnel: rules first, judgement second, people last
- The trigger stayed a person; the reader stayed a person

## TODAY, LIVE — ALL OF IT REAL

- A pipeline around the judge — a graph of steps, in code we read
- Real systems: GitHub Issues, Outlook, an Azure OpenAI deployment
- Four chapters: the spine · unattended · people · the bill
- Two live edits and one break — everything else just runs

You leave able to: decide where planning lives, make a process safe to run twice, and design where people sit in it.

# The judge has a person problem

*Four parts, one loop — and three jobs we quietly kept for ourselves.*

REASONING

MEMORY

TOOLS

PLANNING

Day 1: planning had two ends — the trigger and the stop. Today, the middle: who chooses the steps.

1

## The trigger is a person

Nothing happens until somebody presses enter. A story that lands at 3am waits for morning.

2

## The feeder is a person

Somebody finds the story, copies it in, and asks the question the agent should have been asked already.

3

## The reader is a person

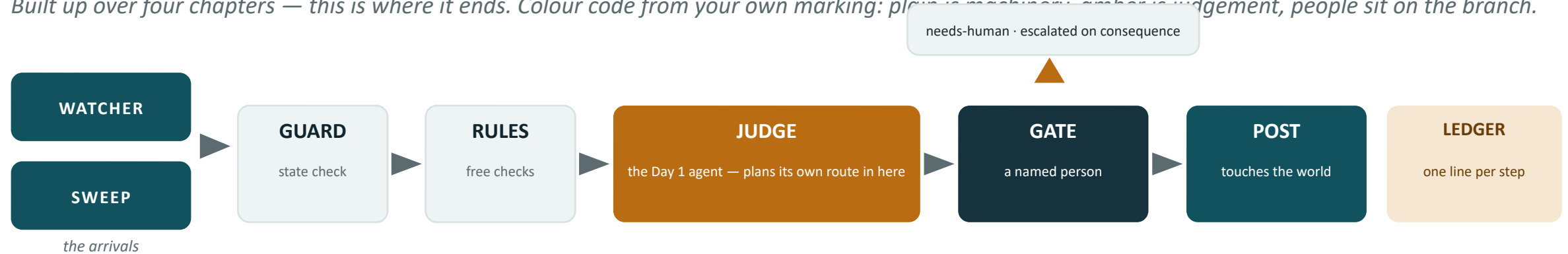
Somebody reads the verdict, copies the comment out, and posts it where the author will see it.

Forty stories a week, fifteen minutes each: ten hours of a senior week spent being the trigger, the feeder and the reader.

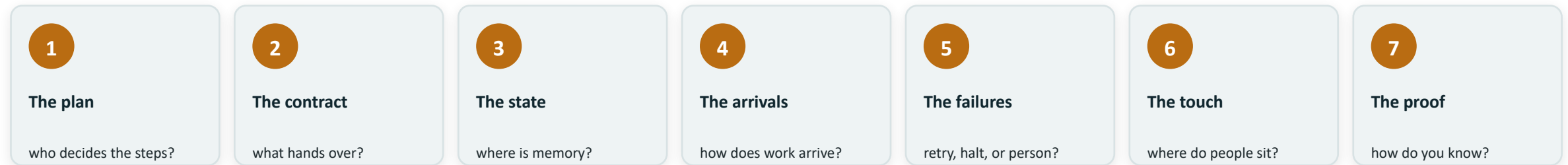
Today's sentence: remove the people without removing judgement.

# The shape of a process that runs itself

Built up over four chapters — this is where it ends. Colour code from your own marking: plain is machinery, amber is judgement, people sit on the branch.



## STATE, ON THE WORK ITEM



Seven decisions. Make them on purpose, or meet them in production.

# Chapter 1 — from a marked process to a running spine

*Watch for four things. Everything on screen is live: the tracker, the model, the comments.*

1

## The agent drafts the plan

Copilot is handed the marked-up process and asked for the steps to run it unattended.

*Agent drafts. Design ratifies. Run executes.*

2

## The ratified plan runs

The graph, read node by node — then STORY-4471 judged on the real tracker, the comment landing on the real issue. STORY-4495 dies free at the rules edge; the model is never called.

*Rules first, judgement second — the funnel, priced.*

3

## Run it twice, on purpose

Machinery steps repeat exactly. The judge returns the same verdict in different words, by a different route.

*Equivalent, not identical — the texture of judgement.*

4

## The ledger grows

One line per step: seconds, tokens, cost. The machinery lines are flat; only the judge's line moves.

*Two planning regimes, one table.*

Note what the pipeline does not yet do: it never writes down what it has done. Hold that thought.

# What you just watched

*The spine runs. Three decisions are now real, and one flaw is armed.*

1

## The plan, in two places

The spine is planned in the design — fixed steps you can test, price, and put a service level on. Inside the amber box the Day 1 agent still plans its own route: its tool calls varied between runs; no machinery line did.

*Planning where variety lives, determinism where liability lives.*

2

## The contract held

Yesterday the judge answered in three sections for a person. Today the next reader is code, so the same three things came back as fields: verdict, findings, comment. However the agent got there — that schema is the only door out.

*Free text between steps is where automation dies.*

3

## Nothing was written down

The comment landed, and the work item still says state:new. The pipeline has no memory of what it has done — which is fine, right up until a second trigger exists.

*Remember this. It bites in about ten minutes.*

The rules edge killed STORY-4495 before the model was called — the first tier of the funnel costs nothing, and the ledger proves it.

# Chapter 2 — nobody at the keyboard

*Two arrivals, one story, one wound, one four-line cure — and a kill test.*

1

## The unattended moment

A story filed from a phone. The watcher wakes, judges it, posts — hands visibly off the keyboard.

*This is the moment the day is named for.*

2

## The collision

The morning sweep runs against the same intake. Two runs, one story: the comment posted twice, on screen, in the real tracker.

*With two triggers, at-least-once is the guarantee you actually have.*

3

## The cure, typed live

A state write on success, and a guard at the top: not new, not ours. Four lines. Then the backfill, by hand — and the sweep re-run ends at the guard.

*Backfilling state is part of adding state.*

4

## The kill test

The worker is killed mid-run and restarted. Finished steps are not redone; the one in flight runs again; nothing doubles.

*A crash is a resume, not a redo.*

A collision is not bad luck. It is an appointment — and we have just watched it kept.



# What you just watched

*Duplicate delivery, idempotency, and the guard — decisions four and five, running.*

1

## Two triggers are why duplicates exist

Every real process grows an event arrival for speed and a schedule as the safety net. The moment it has both, the same work can arrive twice. That is a property of arrivals, not a bug in either trigger.

2

## One state check is why they do not matter

A step is idempotent when running it twice changes nothing the second time. The guard reads the work item's state before working. Every trigger — including ones added next quarter — inherits it, because it lives in the process, not in a trigger.

3

## Failure now has a taxonomy

Transient failures retry — the dial sits on the read, which is safe to repeat. Deterministic failures halt and alert. Judgement failures go to a person. Only idempotent steps ever earn a retry.

The read carries the retry dial; the guard makes the work safe; the crash became a resume. Repeat-safety was demonstrated, not asserted.

# Chapter 3 — people, wired in properly

*The exception path, end to end: escalation, a waiting gate, a clock, and a failure.*

1

## Routed on consequence

The contract grows an escalate flag: card data, money, legal wording — properties known before the run starts. STORY-4512 routes itself to needs-human. Nothing posts.

*What it touches — not how confident it sounds.*

2

## The gate, for real

A mail lands with everything a reviewer needs: verdict, findings, the drafted comment. The run parks — durably. Approve from the phone; it wakes and posts.

*Approve-then-act, on the one step that touches the world.*

3

## The clock

A second gated story is left unanswered. Past the clock, a [STALL] mail lands in the inbox, on screen.

*Every waiting state has a clock and an owner.*

4

## The break

The pipeline's token quietly loses write access. The post step goes red; the run records Failed. Fix the cause, resume — the run picks up at the failed step.

*No re-judge. No second approval. Finished steps stay finished.*

The gate is not a person reading output afterwards. It is the pipeline refusing to act until a named person says so.

# What you just watched

*The touch and the failures — designed, not improvised.*

1

## The exception path is the product

Low-touch is not no-touch. The design spends people only where they change the outcome: consequence routes to a person; everything else flows. The five that reach a human are the five worth a human.

2

## Waiting is a state, not a silence

The parked run cost nothing while it waited — no loop, no tokens, just a checkpoint. And because every wait carries a clock and an owner, an unanswered approval turned itself into somebody's mail.

3

## The failure matched its response

A wrong permission is deterministic: no retry dial fixes it, so the run halted red and said where. One fix, one resume, and the pipeline finished the run without repeating the judgement or the approval.

Nothing stalls silently. That is the standard — and it took one small sweep and one clock to meet it.

# Chapter 4 — one story, end to end

*Everything at once, from the phone to the posted review — then the bill.*

1

## Filed from the phone

A fresh, well-formed story. The watcher picks it up with nobody at the laptop.

*The 3am story, at three in the afternoon.*

2

## The seven decisions, live

Guard, rules, judge, gate, post — each decision called out in the second it fires on screen.

*The map slide, running.*

3

## One human minute

One mail, one word — Approve — and the review posts itself. The person did the only part that needed a person.

*That is the touch, engineered.*

4

## The bill

The ledger the pipeline has been writing all session, totalled: cost per accepted outcome, from today's real runs.

*Not an estimate. The invoice.*

Watch how little of this run is judgement — and how the price follows exactly that shape.

# The bill, read off the ledger

*Low-touch is cost engineering. The funnel is the invoice.*

## THE COST LADDER — PER STORY

|                       |                                    |
|-----------------------|------------------------------------|
| <b>rules</b>          | nothing — code, running on nothing |
| <b>judgement</b>      | pennies — tokens, metered per run  |
| <b>a human minute</b> | the line that dwarfs both          |

*So the design pushes every story down the ladder as far as trust allows — and spends the dear line only where it changes the outcome.*

## FORTY STORIES A WEEK

|           |                                          |
|-----------|------------------------------------------|
| <b>12</b> | die free at the rules edge               |
| <b>28</b> | judged, for pennies                      |
| <b>5</b>  | reach a person — the five worth a person |
| <b>23</b> | post themselves                          |

**The ten-hour review week becomes seventy-five minutes.**

Cost per accepted outcome — computed on screen, from the ledger, not from a vendor's slide.

# How this goes live

*You never decide to trust it — you run out of reasons not to.*

## SHADOW

Runs on real stories; nobody sees its output but you.

*Exit: agreement with what reviewers actually did, on thirty real cases.*

## SUGGEST

Drafts are visible; a person posts every one.

*Exit: the reviewer stops editing the drafts.*

## ASSIST

Posts routine verdicts itself; consequence still gates.

*Exit: a quarter of exception-path audits come back clean.*

## ACT

Runs the path alone; people hold the escalations and the samples.

*Standing: the ledger, the stall sweep, and a kill switch.*

TODAY'S PIPELINE SHIPS HERE

Each move to the right is earned by measured agreement — and the ledger is the measurement.

# The same seven, in boxes

*Copilot Studio · Power Automate · n8n · Temporal — every platform is a skin over what ran today.*

- |                         |                                                                                                                                                         |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1 • the plan</b>     | A workflow designer fixes the plan in the design; an agent framework frees it into the run. Real systems place the boundary — exactly as you did today. |
| <b>2 • the contract</b> | Structured output, output mapping, typed variables between nodes.                                                                                       |
| <b>3 • the state</b>    | List columns, run context, labels — the work item remembers, whatever it is called.                                                                     |
| <b>4 • the arrivals</b> | The triggers page: event, schedule, manual — and the same duplicates when you wire two.                                                                 |
| <b>5 • the failures</b> | Retry policies, run-after settings, error branches — the taxonomy, as dropdowns.                                                                        |
| <b>6 • the touch</b>    | Approvals connectors, human-in-the-loop nodes, waiting runs.                                                                                            |
| <b>7 • the proof</b>    | Run history, analytics tabs, billing meters — the ledger, sold back to you.                                                                             |

You now know what the boxes are for — which is the part no platform can teach you.

# Three things to carry

*If the rest of the day fades, keep these.*

1

## **Planning where variety lives, determinism where liability lives**

The spine is designed; the judgement plans inside its box; the boundary moves only when the work's shape does.

2

## **Anything with two triggers must be safe to run twice**

Duplicates are an appointment, not bad luck. State on the work item and a guard at the top — every trigger inherits it.

3

## **Nothing waits without a clock and an owner**

A parked run is fine. A silent one is not. The stall sweep turns waiting into somebody's mail.

All three are one habit: decide on purpose what production would otherwise decide for you.



# Before Day 3

*Twenty minutes, and it makes the next session concrete rather than abstract.*

## MARK THE EXCEPTION PATHS

- Take the process you brought yesterday
- Mark what escalates — and on what consequence
- Mark what waits — and who owns each clock
- Mark what may never run twice

## BRING ONE API

One API your team owns — its contract, or even just its base URL and what it does. Tomorrow, agents write its tests: from the contract, against a seeded break, with the failures triaged.

The messier the API, the better the session.

A process this disciplined still ships broken stories if the tests behind it are thin. That is Day 3's problem.

# Questions

*And anything the chapters raised that we ran past.*

## IF THE ROOM IS QUIET, START HERE

- Which of your processes has two triggers today — and where is its guard?
- Where would your judge's escalate flag fire — what does it touch?
- Which waiting state in your world has no clock and no owner right now?
- What would cost per accepted outcome be for the process you brought?

Thank you. The repo, the runbook and every command from today travel with the materials.